

DNA Data Storage Encoder

Implementation, evaluation,
testing and web demo

9 October 2026

480 trials	303 tests	270 trials
Matched-budget codec validation	Automated checks, 93% line coverage	Error-tolerant web codec benchmark

Executive summary. This project converts opaque file bytes into indexed DNA oligonucleotides and reconstructs the file from clean or simulated sequencing reads. It combines several payload mappings with a common container, index, integrity checks, consensus decoder and outer error correction. A matched-budget experiment runner now compares codecs at exactly equal total synthesized nucleotide counts.

Five fresh examples were run before preparing this report. Text and binary files were recovered exactly; a binary file was also recovered after two oligos were removed and under a 3% insertion/deletion/substitution (IDS) model at depth 12. A 9% IDS example at depth 2 failed with an incomplete header and returned no recovered data.

The earlier 480-trial validation and completed 100 KB clustering experiment provide broader implementation evidence. They do not establish a general advantage for any codec, and the full publication study remains unfinished.

An adversarial testing campaign then ran 303 Python tests and 10 browser-codec tests. Across corrupted reads, mixed pools, truncated archives and misuse, no decoder ever reported success with wrong bytes, but 10 reproducible defects were found and are documented as failing tests. A public web demo (<https://dadabsdk.github.io/dna-encoder/>) adds an error-tolerant browser format. In 270 seeded trials it recovered 10 of 10 files exactly at 1% per-base error with three reads per strand and 25% parity, and returned partial files with flagged uncertain bytes when damage exceeded the parity budget.

Evidence boundary. Fresh examples are single realizations, not estimates of recovery probability. Historical experiments are identified separately. All successful fresh examples require both the decoder's integrity checks and exact input/output byte and SHA-256 equality. No physical DNA synthesis experiment was performed here.

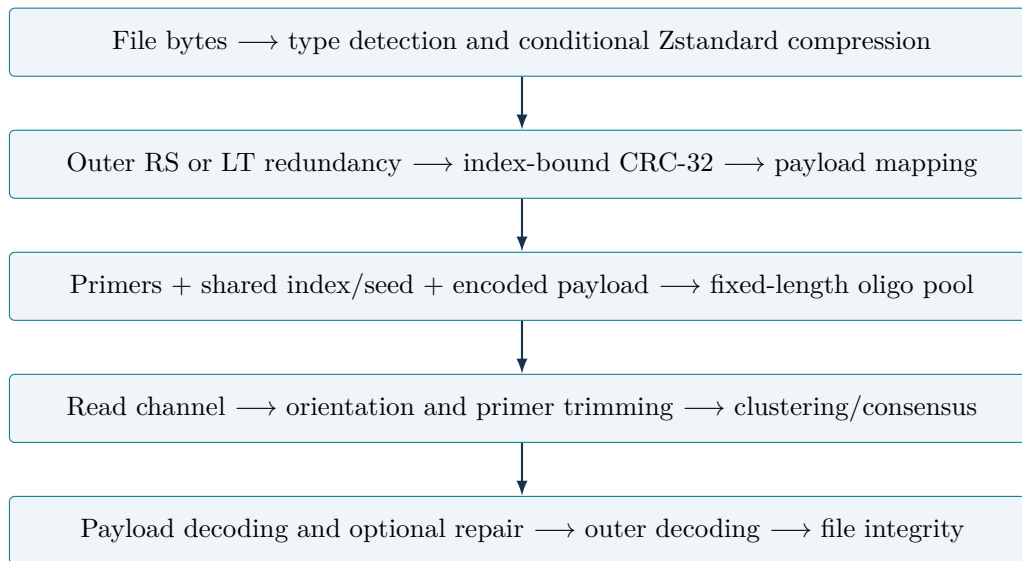
Contents: architecture; physical format and metrics; worked examples; matched-budget validation; clustering at scale; earlier research phases; testing campaign; web demo; limitations and next steps; reproduction and evidence.

1 Project objective and architecture

Question: do sequence constraints justify the nucleotides they consume?

The central design question is whether spending nucleotides on sequence constraints improves file recovery more than spending the same budget on additional error correction. The implementation is an integration and trade-off study. It does not claim to invent rotating ternary mapping, steering redundancy, Reed–Solomon coding or LT fountain coding.

End-to-end workflow



Payload mappings

Mapping	Role and implementation
Naive 2-bit	Quaternary mapping; optional whitening creates the unconstrained whiten+RS comparison.
Goldman	Rotating ternary mapping. Uses this project's common ECC, not the original historical overlap scheme.
Steering A	Quaternary information symbols with inserted steering positions and seed retries.
Steering B	Rotating ternary information symbols with steering; lower raw information rate.
Steering C	Run-length-limited quaternary information mapping with steering and a capacity margin.
Fountain	Screened LT droplets replace outer RS. CRC and file integrity checks remain in place.

Code organization

`encoder.py` and `decoder.py` orchestrate the pipeline. The `codecs/` package contains mappings; `ecc.py` implements CRC/RS; `container.py` handles compression and metadata. `channel.py` provides controlled IDS simulations; `nanopore.py` integrates external channels. `benchmark.py` performs budget matching and repeatable evaluation.

2 Physical format, protection and metrics

All comparisons must account for the complete physical pool.

Default oligo layout

Forward primer	Index	Seed	Payload region	Reverse-primer tail
20 nt	15 nt	3 nt	142 nt	20 nt

The total is 200 nt. The physical tail is the reverse complement of the reverse primer. The codec-independent index and seed use rotating ternary encoding. Header oligos use reserved indices below 64 and a fixed bootstrap format, allowing recovery of codec parameters before data-payload decoding. The default uses five distinct copies of each header fragment; the number of fragments depends on codec metadata.

Integrity and recovery

Each data frame has a 32-bit CRC bound to its index. In the default geometry, whitened 2-bit mapping carries 252 data bits plus a 32-bit CRC in 142 nt; Goldman carries 193 data bits plus the same CRC. The CRC's nucleotide cost depends on the mapping.

For RS arms, bytes form rows of width $\lceil d/8 \rceil$, with d payload bits per oligo. RS columns are coded over GF(256), with at most 255 symbols per block. Rows are bit-packed into oligos, so a missing physical oligo can cross row boundaries. In the fresh examples, the parity rule is $p = \lceil 0.15k \rceil$; parity rows and physical parity oligos are therefore not interchangeable counts.

Erasure strength. CRC-failing payloads are discarded; the outer RS path uses erasures and its consistency checks.

Repair strength. Adds bounded, CRC-guided homopolymer run-length edits and outer RS errors-and-erasures decoding. It is the CLI default. Both strengths are evaluated on identical read sets in this report.

Decompression is followed by original-length and file CRC-32 verification. For the worked examples, the caller additionally checks byte equality and SHA-256. These checks are software integrity measures, not cryptographic authentication of a DNA pool.

Density definitions

For N fixed-length oligos of length L , the charged synthesis budget is

$$B_{\text{nt}} = \sum_{i=1}^N L_i = NL.$$

The whole-pool densities reported by this implementation are

$$D_{\text{file}} = \frac{8 n_{\text{original}}}{B_{\text{nt}}}, \quad D_{\text{stored}} = \frac{8 n_{\text{stored}}}{B_{\text{nt}}}.$$

The separate code-density metric divides stored bits by the data rows' share of payload-region nucleotides; that denominator includes frame CRC, row padding and steering. Its accounting is defined in `dnastore/metrics.py`. Tiny files are dominated by header overhead. Compression can improve file density without improving the DNA mapping.

3 Fresh worked examples: recovery outcomes

Executed for this report using `scripts/report_examples.py`.

Inputs and encoder settings

The text example contains 205 UTF-8 bytes, including accented Latin and Devanagari text. Goldman encoding compresses it to 180 stored bytes. The binary input is 1,024 deterministic pseudorandom bytes generated with NumPy seed 20261008 and uses whitened 2-bit mapping. Compression is not selected for that input. Both pools use global encoder seed 42, 200-nt oligos, default primers, parity ratio 150 per thousand and one worker.

Quantity	Text pool	Binary pool
Original / stored bytes	205 / 180	1,024 / 1,024
Header / data-plus-parity oligos	10 / 11	15 / 38
Total oligos / nucleotides	21 / 4,200	53 / 10,600
RS data rows / parity rows	8 / 2	32 / 5
Whole-pool file density (bits/nt)	0.3905	0.7728
Oligos with hard-constraint violations	0	35

The binary pool is deliberately an unconstrained baseline. Its 35 violating oligos are recorded, not hidden. The examples use the research API; the CLI would require `-allow-violations` to write that pool. This does not weaken CRC or file checks.

Observed outcomes

ID	Read condition	Bytes	Reads	Erasure	Repair
E1	Text, noiseless	205	21	Pass	Pass
E2	Binary, noiseless	1,024	53	Pass	Pass
E3	Two oligos removed	1,024	51	Pass	Pass
E4	3% IDS, depth 12	1,024	636	Pass	Pass
E5	9% IDS, depth 2	1,024	106	Fail	Fail

Pass means exact reconstruction of the original bytes. Both strengths use the same reads in each row. In E3, indices 64 and 65 were removed; all header oligos were retained. Thus E3 demonstrates recovery of two data-oligo losses, not arbitrary loss of two oligos anywhere in the pool.

For E4 and E5, coverage is lognormal with $\sigma = 0.27$, orientation is reversed with probability 0.5, and there is no explicit dropout term. Read allocation is multinomial at a fixed total count. E4 has $p_s = p_i = p_d = 0.01$, depth 12, channel seed 101. E5 has $p_s = p_i = p_d = 0.03$, depth 2, channel seed 202.

Failure is part of the demonstration. E5 reports **header incomplete**. The decoder returns no reconstructed file, so data decoding is not established for that case. Changing both depth and error rate does not identify a recovery threshold or isolate either factor's causal effect.

5 Recorded matched-budget validation

Historical run: validation_optimized_20261008; not rerun for this report.

This run uses two independent synthetic payload seeds, each 800 bytes, and six arms. All arms have exactly 62 oligos of 200 nt: **12,400 total nt per arm**. Each payload/arm receives 20 independent IDS channel realizations at mean depth 8, with $p_s = p_i = p_d = 0.01$ and lognormal coverage $\sigma = 0.27$. Each read set is decoded at both strengths: $2 \times 6 \times 20 \times 2 = 480$ decoder trials, corresponding to 240 read sets. All 12 encoded pools pass noiseless reconstruction. [P3]

Arm	Payload seed 0		Payload seed 1	
	Erasure	Repair	Erasure	Repair
Whiten + RS	20/20	20/20	20/20	20/20
Goldman	20/20	20/20	20/20	20/20
Steering A, P=8	20/20	20/20	20/20	20/20
Steering C, P=8	20/20	20/20	20/20	20/20
Steering B, P=6	17/20	20/20	19/20	19/20
Fountain	20/20	20/20	20/20	20/20

Uncertainty and interpretation

The 95% Wilson intervals for the counts appearing above are:

Success count	Estimated recovery	Wilson 95% interval
20/20	100%	83.89–100.00%
19/20	95%	76.39–99.11%
17/20	85%	63.96–94.76%

For k successes out of n trials and $z = 1.96$, the interval is

$$\frac{\hat{p} + z^2/(2n) \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \quad \hat{p} = k/n.$$

Twenty successes do not prove a 100% population recovery probability. Intervals are conditional on each fixed payload and configuration; shared-payload trials are not pooled into a single confidence interval across payloads.

Repair increases steering-B recovery from 17/20 to 20/20 on payload 0, but does not improve its 19/20 outcome on payload 1. The other five arms succeed in every tested condition here. This is evidence of correct operation and a finite-sample difference at one depth, not a general ranking of coding strategies.

Budget matching matters. Extra parity is assigned to denser mappings until all physical pool sizes match. Header differences, padding and primers are included; no unused filler oligos are added. Global matching lets the lowest-rate arm influence the common budget. Pairwise steering-versus-whiten+RS experiments are still needed across periods, weights, payload types and channel settings.

6 Recorded 100 KB clustering-scale experiment

Historical Phase 3 run; single payload and unmatched synthesis budgets.

The completed run encodes 100,000 bytes separately with six arms, producing thousands of oligos per pool. It measures all distinct-body pairs, clusters simulated reads and attempts complete file recovery at requested mean depth 10. Both calibrated Badread and a controlled IDS9 channel are evaluated. [P4]

Arm	Oligos	Min. body ED/ L	IDS9 split (%)
Whiten + RS	3,678	0.3812	2.48
Goldman	4,786	0.3250	5.81
Steering A, P=8	4,169	0.3750	2.50
Steering C, P=8	4,325	0.3688	2.45
Steering B, P=6	5,742	0.3187	6.41
Fountain	3,735	0.3625	2.04

$L = 160$ nt is the body length after removing primers. The minimum distances are measured over all pairs of distinct oligo bodies. No pair is within the $0.30L$ read-clustering radius. This finite-pool observation is not a proof about arbitrary future pools or error realizations.

Read grouping and file recovery

- All 12 arm/channel rows report zero observed cluster merges and zero `groups_multi_index` after decoding.
- Badread has zero measured split rate; all six arms recover at both decoder strengths.
- IDS9 splits between 2.04% and 6.41% of observed oligos across multiple multi-read groups. No arm recovers the entire file at either strength.

Here IDS9 uses **4% substitutions, 2.5% insertions and 2.5% deletions**. It differs from the equal-thirds 9% IDS model used in fresh example E5. The scale run uses lognormal coverage spread 0.27 and requested mean depth 10. Pool sizes and synthesis budgets are not equal across arms; it is a clustering check, not a fair density-versus- recovery comparison.

Channel scope

The current project design uses identity-matched Badread as the main read-level channel and uniform IDS as a controlled reference. The raw-signal squigulator–Dorado path is retained as a stress test because the project’s calibration found it miscalibrated against available real R10.4.1 storage data. [P5]

The real calibration source consists of plasmid fragments rather than a synthesized 200-nt oligo pool. Badread does not reproduce the small systematic inter-read component seen in those data. Consequently, simulated recovery is not a wet-lab performance guarantee. This report summarizes the recorded project calibration; it does not independently revalidate the external dataset or literature.

7 Earlier research phases: constraints, structure and channel

Phase 2–3 measurements. The density sweep predates the format-v3 inner CRC-32 and is not directly comparable with v3 results.

What sequence constraints cost (Phase 2 sweep, format v2)

Each arm encoded the same input into 200-nt oligos. Code density counts payload bits per payload nucleotide; effective density counts file bits per synthesized nucleotide, including primers, index, header oligos and parity. A violating oligo breaks at least one hard synthesis constraint (homopolymer, GC content or primer match).

Arm	Code density (b/nt)	Effective density (b/nt)	Max run	Violating oligos
Quaternary, raw	1.882	1.074	132	87.8%
Quaternary, whitened	1.882	1.074	8	86.3%
Goldman rotating ternary	1.463	0.851	3	3.4%
Steering A, P=8	1.668	0.930	3	0.0%
Steering C, P=8	1.619	0.884	3	0.0%
Steering B, P=6	1.246	0.714	3	0.0%

Unconstrained quaternary mappings are densest but most of their oligos break hard constraints; whitening alone does not fix this. Every constrained arm met the hard constraints at a measurable density cost. Whether that cost buys more recovery than the same nucleotides spent on parity is the open question of the study.

A hidden structure cost of homopolymer limits

Median minimum free energy (kcal/mol, 95% CI) of 2,000 random 160-nt bodies; more negative means more stable self-folding.

Sequence model	MFE, 37 °C	MFE, 60 °C	Paired fraction
Whitened quaternary (i.i.d.)	-17.2 [-17.4, -17.0]	-8.8 [-9.0, -8.6]	0.52
Rotating ternary (run length 1)	-27.1 [-27.3, -26.9]	-14.7 [-14.9, -14.6]	0.63
Control: different banned successor	-25.0 [-25.2, -24.8]	-13.5 [-13.6, -13.3]	0.59

Run-length-1 coding folds markedly more stably. Banning one successor raises the probability of reverse-complementary k-mer pairs, and a control that bans a different successor reproduces most of the gap. Constrained alphabets therefore start from a structural deficit that their steering must repay.

Choosing a structure metric

Spearman correlation with equilibrium primer-site accessibility at 60 °C on a held-out half (n = 505):

Proxy	ρ
Incremental k-mer proxy (tier 1)	-0.373
Local partition function, window 80 nt (tier 2)	+0.778
Global MFE at 60 °C	+0.207

Global MFE is a poor surrogate for primer accessibility, so accessibility became the primary structure metric; the cheap k-mer proxy is only a tie-breaker.

Read-channel calibration against real nanopore data

Real reads: public R10.4.1 data (Chen et al. 2025, Zenodo 10.5281/zenodo.16883332), Dorado basecalls on identical reference intervals.

Measure	Real hac	Real sup	Squigulator → Dorado	Badread (matched)
Per-base error	1.51%	1.01%	9.12%	1.55%
Sub. / del. / ins.	0.67 / 0.57 / 0.27%	–	4.7 / 3.0 / 1.4%	0.67 / 0.58 / 0.30%

Identity-matched Badread reproduces the real error level and is the main read-level channel; squigulator is retained only as a labelled, miscalibrated stress channel. Badread omits the systematic read-to-read errors seen in real data.

8 Testing campaign and known defects

Full suites rerun for this report; evidence in [results/testing_20261009/](#).

Python tests (pytest)	303
passed / skipped / failed	292 / 1 / 0
known defects (strict expected failures)	10
Browser-codec tests (Node.js) passed / failed	10 / 0
Line coverage of <code>dnastore/</code>	93%

What was attacked. Heavy insertion/deletion/substitution noise, truncated and chimeric reads, junk-only input, pools mixing two files with the same primers, outer Reed–Solomon erasures at and beyond capacity (checked against the `reedsolo` reference), undetected corrupted oligos, 556 corrupted or truncated stream archives, 33 malformed FASTA/FASTQ cases, determinism across hash seeds and worker counts, and the local web server’s host, token and path-traversal checks. 7 guard tests now pin these properties. **No test produced a successful decode with wrong bytes.**

Defects found. Each is a strict expected-failure test in `tests/test_adversarial.py`: the suite stays green, and a fix makes the test pass, which then fails the run until the marker is removed. None is fixed yet.

ID	Defect
B1	a config file without <code>primers:</code> resolves the default <code>primers.yaml</code> next to the config instead of the bundled primers
B2	non-integer <code>oligo_len</code> escapes the CLI as a <code>TypeError</code> traceback
B3	<code>reedsolo</code> is imported by the default <code>repair</code> decoder (<code>ecc._ee_column</code>) but is only a [test] extra
B4	<code>_ee_column</code> trusts <code>reedsolo</code> ’s global GF tables when <code>gf_exp[1] == 2</code> , so another primitive polynomial silently breaks it
B5	fountain encoder never checks decodability; with default overhead most small pools ($k \sim 7$) fail even from perfect reads
B6	gzip is detected by lowercase <code>'gz'</code> suffix only
B7	truncated gzip raises <code>EOFError</code> , which the CLI does not catch
B8	web accepts unbounded <code>mean_coverage</code> ; one request can exhaust host memory in the read simulator
B9	web allows <code>oligo_len</code> 100, where the header of 5 of 6 codecs needs more than 15 fragments
B10	cluster grouping takes the first-seen header fragment, so a minority contaminant pool can block recovery of the majority file

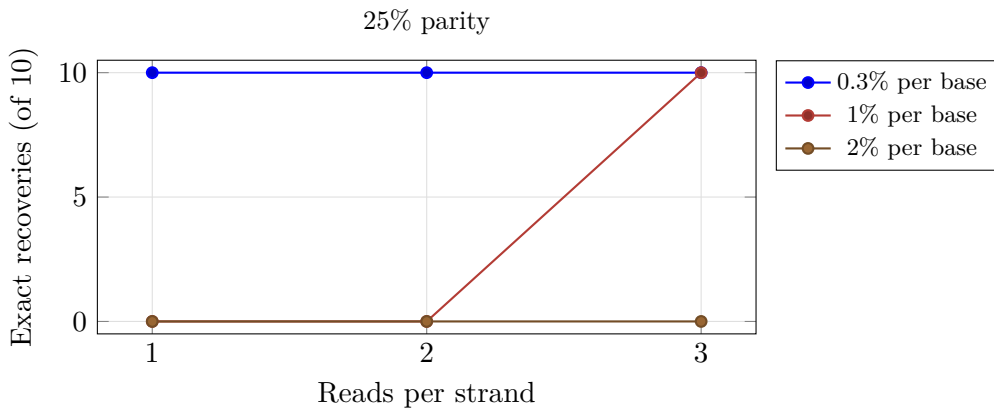
Impact. The fountain-codec defect affects benchmark interpretation for small files: a pool that cannot be decoded from perfect reads makes later channel trials meaningless. The benchmark records `noiseless_ok` per pool, which must be checked. The `reedsolo` defect breaks the default CLI decoder on a plain install.

9 Web demo and the error-tolerant browser codec

<https://dadabsdk.github.io/dna-encoder/>; benchmark: scripts/site_resilience.mjs.

The static site runs entirely in the browser and is deployed by GitHub Actions with commit-versioned asset URLs. Its default format (DNASTORE-RS-1) is separate from the research codec: 32-byte payloads become 160-nt strands [index || payload || CRC-32] with keystream whitening, plus Reed–Solomon parity strands across each block of up to 255 strands and eight copies of the metadata. The decoder (i) accepts reads whose CRC verifies, (ii) repairs one substitution, insertion or deletion per read by CRC-guided search, (iii) merges several reads of one strand by banded-alignment consensus, (iv) rebuilds missing strands from parity, and (v) beyond the parity budget returns the file with unrecoverable bytes reported as uncertain. Bytes not reported uncertain are exact; the benchmark asserts this in every trial.

Benchmark. 270 trials: random 20,000-byte payloads, 10 seeds per cell; 5% strand dropout; per-base error split 50% substitution, 25% insertion, 25% deletion, independent per read. Synthesis cost: 5.82 / 6.58 / 7.82 nt per byte at 10 / 25 / 50% parity (raw 2-bit mapping: 4 nt per byte).



Exact recoveries by parity level.

Error/base	Reads	10% parity	25% parity	50% parity
0.3%	1	0/10	10/10	10/10
0.3%	2	10/10	10/10	10/10
0.3%	3	10/10	10/10	10/10
1%	1	0/10	0/10	0/10
1%	2	0/10	0/10	10/10
1%	3	1/10	10/10	10/10
2%	1	0/10	0/10	0/10
2%	2	0/10	0/10	0/10
2%	3	0/10	0/10	0/10

Partial recovery at 25% parity.

Err.	Reads	Exact	Guaranteed	Correct	ms
0.3%	1	10/10	100.0%	100.0%	27
0.3%	2	10/10	100.0%	100.0%	24
0.3%	3	10/10	100.0%	100.0%	33
1%	1	0/10	49.9%	69.9%	66
1%	2	0/10	78.8%	87.8%	171
1%	3	10/10	100.0%	100.0%	205
2%	1	0/10	15.7%	43.4%	98
2%	2	0/10	33.5%	61.9%	352
2%	3	0/10	58.2%	76.1%	553

Guaranteed is the mean share of bytes not flagged uncertain; *Correct* also counts flagged bytes whose best guess was right; ms is the median decode time (Node.js v22.22.0).

Multiple reads per strand matter more than parity: a single read at 1–2% error per base usually carries two or more errors, which single-edit repair cannot fix. The channel is synthetic with independent errors; real systematic errors and PCR bias would lower these numbers. This browser format is a teaching and robustness demonstration, not a replacement for the constrained research codec, and is not yet readable by the CLI.

10 Engineering quality and remaining limitations

The software workflow is operational; the broader research study is not complete.

Completed reliability work

- Added the missing matched-budget benchmark command, independent channel seeds, both decoder strengths, Wilson intervals, empirical depth search, output tables, plots, source snapshots and provenance records.
- Added multiline FASTA, wrapped FASTQ and gzip support with malformed-record checks. Ambiguous reads containing N are discarded and counted; lowercase ACGT is normalized.
- Validated key RS, header, geometry, probability and codec parameters. Made the strict CLI constraint policy apply to baseline codecs as well.
- Bundled default primers so installed-wheel roundtrips work outside the checkout.
- Excluded groups composed entirely of verified header candidates from futile data-repair attempts, while preserving uncertain and mixed header/data clusters.

The prior expanded full suite passed **240 tests with one skip**. After the header optimization, the affected suites passed 44 tests; header-only and mixed-cluster regressions also passed separately. These are overlapping verification runs, not additive test counts. Dependency consistency, wheel construction and an installed-wheel roundtrip were also checked. [P1]

A saved single-case profile showed repair attempts falling from 17 to 5 and profiled time from 9.32 to 3.53 seconds. Across 120 shared before/after trials, recovery and continuous error metrics were identical. Timing was measured with concurrent background load; it is not a general throughput claim.

Limitations

1. **Evidence size.** Fresh examples use one seed per noisy setting; the matched validation uses only two small synthetic payloads at one depth.
2. **Physical fidelity.** IDS is a controlled model. Badread lacks systematic inter-read errors. Neither proves synthesis, PCR or long-term storage reliability.
3. **Constraint costs.** Baselines may emit violating sequences in research mode. The synthesis-side penalty of these violations is not established by this report.
4. **Inference.** The bisection depth result is an empirical boundary. Independent stochastic outcomes need not be monotone; right censoring and observed reversals must be reported.
5. **Operations.** Output directories are protected from overwrite. Complete depth probes are saved incrementally; automatic interrupted-run resume is not implemented.

Next steps. Run the larger `configs/paper.yaml` study, add a licensed real-file corpus, sweep steering periods and normalized weights, and perform the specified coverage/accessibility sensitivities with explicit assumption labels. Complete literature verification before making novelty or publication-level claims.

11 Reproduction and evidence manifest

All paths below are relative to the project root unless stated otherwise.

Reproduce the fresh demonstrations

```
.venv/bin/python scripts/report_examples.py \
--out results/report_examples_rerun
```

The output directory must be new or empty. The script saves original files, physical pools, exact read sets, successful reconstructions, simulator counts, decoder reports, configuration, SHA-256 digests and source checksums. E5 intentionally has no recovered file.

Run the benchmark and compile this document

```
.venv/bin/dnastore benchmark configs/benchmark_validation.yaml \
--out results/phase4/validation_rerun
.venv/bin/dnastore benchmark configs/paper.yaml \
--out results/phase4/full_study
.venv/bin/python scripts/build_project_report.py
pdflatex -interaction=nonstopmode -halt-on-error \
-output-directory=output/pdf reports/dna_storage_project_report.tex
pdflatex -interaction=nonstopmode -halt-on-error \
-output-directory=output/pdf reports/dna_storage_project_report.tex
```

This source is standalone: diagrams and chart coordinates are embedded; no external images or bibliography files are required. A standard LaTeX installation with TikZ, PGFPlots, Latin Modern, microtype and the listed table/layout packages is sufficient. The full study can be substantially more expensive than the demonstrations.

Rerun the tests and the web-codec benchmark

```
.venv/bin/pytest -q --cov=dnastore --cov-report=json:OUT/coverage.json \
--junitxml=OUT/pytest_junit.xml
node --test --test-reporter=tap site/tests/resilient.test.mjs
node scripts/site_resilience.mjs results/site_resilience_rerun
```

Primary project evidence

- P1** docs/PROJECT_STATUS.md and docs/DESIGN.md: design decisions, completed engineering work and recorded verification history.
- P2** results/report_examples_20261008/examples.json and results.csv in the same directory: fresh examples, complete digests and decoder reports.
- P3** results/phase4/validation_optimized_20261008/: summary.csv, trials.csv, pool manifests and provenance.json; historical 480-trial validation.
- P4** results/phase3/cluster_scale_100KB/cluster_scale.csv: all 12 historical scale-run arm/channel rows.
- P5** docs/notes/channel_calibration.md: calibration methods, recorded channel measurements and transfer limitations.
- P6** docs/BENCHMARK.md, dnastore/metrics.py and dnastore/benchmark.py: budget, metric and statistical definitions.

- P7** `results/phase2/sweep_v2/summary.csv`, `results/phase2/composition/mfe_summary.csv` and `docs/notes/structure.md`: historical constraint, structure and proxy results.
- P8** `results/testing_20261009/`: JUnit XML, coverage JSON and Node TAP output of the test rerun; defects parsed from `tests/test_adversarial.py`.
- P9** `results/site_resilience_20261009/`: per-trial `results.csv`, `summary.csv` and `provenance.json` (codec SHA-256 recorded).

Traceability. The report builder reads the saved JSON/CSV evidence rather than rerunning or fabricating measurements. `reports/report_evidence.json` records SHA-256 hashes of the evidence files and generated LaTeX source. Rebuilding with changed evidence requires reviewing the interpretation as well as the numbers.